

1-Investigação do incidente (antes da solução). Diante do quadro acima, descreva passo a passo COMO você investigaria — sem pular para a correção. Liste suas hipóteses iniciais (ordene por probabilidade), o que olharia primeiro (quais logs, métricas, traces, comandos ou dashboards) e como cada observação confirmaria ou descartaria uma hipótese. Qual seria seu primeiro sinal de que está no caminho certo?

Resposta 1

Hipóteses

1. Mudança recente — deploy, configmap, feature flag ou atualização de biblioteca alterou o comportamento do consumer, produtor ou infraestrutura.
2. Hot partition / poucas partições — O tópico tem partições insuficientes para 3x o tráfego, ou uma partição específica está saturada.
3. HPA limitado + rebalanceios frequentes — O Horizontal Pod Autoscaler não está escalando adequadamente (maxReplicas baixo, recurso insuficiente, métrica indisponível) OU o consumer está sofrendo rebalanceios constantes por timeout/GC/config.
4. Downstream lento causando backpressure — Antifraude, extrato ou conciliação estão lentos, travando o commit de offset do consumer.
5. Falha no broker Kafka — Disco cheio, líder de partição indisponível, ou problema de rede no cluster Kafka.

Pasos para a confirmação/descarte

1. Contexto e mudanças recentes. Executo `kubectl config current-context` para confirmar o cluster. Depois `kubectl get events --sort-by=.metadata.creationTimestamp | tail -50`. Se vejo `FailedGetResourceMetric` ou `ScalingLimited`, reforça H3. Se vejo `CrashLoopBackOff`, investigo. Então `kubectl rollout history deployment/<consumer>` — rollout recente reforça H1; sem rollout há dias descarta H1 parcialmente. `kubectl describe deployment`

- `<consumer>` — `replicas/maxReplicas` baixos reforça H3. `kubectl describe configmap <kafka-config>` — parâmetros alterados (`max.poll.interval.ms`, `session.timeout.ms`, `enable.auto.commit`) reforça H1.
2. Infraestrutura do Kafka. Uso `kafka-consumer-groups --bootstrap-server <broker> --group <group> --describe` para ver lag por partição. Lag homogêneo em todas → H3/H4. Lag em uma partição específica → H2. Partição sem líder → H5. Complemento com `kubectl get pods -n kafka` e logs dos brokers. Broker caído ou disco cheio → H5.
 3. Saúde do consumer e do cluster. `kubectl describe hpa <consumer>` — `currentReplicas` VS `maxReplicas`, condições de escala. `kubectl top pods` — CPU/mem dos consumers. Logs do consumer com `kubectl logs <pod> --tail=100 | grep -iE "rebalance|commit failed|timeout|gc"`. Rebalanceios frequentes ou resources no limite → H3. Commit falhando com HPA ok → H4. `kubectl top nodes` — cluster sem capacidade → H3.
 4. Saúde do downstream. Consulto métricas de latência p99 e error rate do antifraude e extracto. Downstream lento (p99 elevado) ou com erros → H4. Saudável → descarta H4.

Primeiro sinal de que estou no caminho certo:

Se no passo 1 encontro um rollout recente com alteração de configuração crítica (ex: `max.poll.interval.ms` reduzido) ou um evento `ScalingLimited`, sei que estou na direção certa — uma causa identificável e com ação clara.

2 - São 03:10 e o lag cresce. O que você faz PRIMEIRO para estabilizar (mitigação) e o que deixa para depois (correção definitiva)? Justifique o trade-off entre estancar o problema agora e resolver a raiz, e explique o risco de cada escolha.

Resposta 2

Antes de qualquer ação de mitigação, é conveniente descartar a hipótese H4 consultando as métricas de latência p99 e error rate do antifraude. Se o downstream está saudável, escalaria o consumer sem risco de piorar. Se estivesse lento, escalar o consumer apenas sobrecarregaria ainda mais — nesse caso, a mitigação seria escalar ambos proporcionalmente. Escalar o consumer horizontalmente com

`kubectl scale deployment <consumer> --replicas=<N>`, onde N é definido pela taxa de crescimento do lag: x2 se for linearmente, x3 se for exponencialmente. Paralelamente, se há pods com erro ou em loop de rebalance, executo `kubectl delete pod <pod>` para que o k8s os recrie — o `podAntiAffinity` caso existente no deployment garante que caiam em nodos distintos sem necessidade de intervenção manual nos nodes. O risco dessa mitigação é consumir recursos do cluster que talvez não estejam disponíveis, causando contenção com outros serviços. Por isso monitoro `kubectl top nodes` durante e após o scale. A correção definitiva fica para depois do pico seria revisar a configuração do HPA (`maxReplicas`, métricas), ajustar partições do tópico Kafka, corrigir `configmaps` (`max.poll.interval.ms`, `session.timeout.ms`), e criar testes de carga que repliquem o cenário de 3x o tráfego para validar as mudanças. O trade-off é claro: às 03:10 o objetivo é parar o sangramento com uma ação que funcione para múltiplas hipóteses, mesmo que não ataque a causa raiz. Tentar corrigir a raiz agora arrisca prolongar o incidente enquanto o lag e os duplicados crescem.

3 - O antifraude está recebendo eventos duplicados. Explique as causas prováveis nesse tipo de pipeline (produção idempotente, reprocessamento, particionamento, commit de offset) e como você garantiria idempotência e ordem por transação de forma robusta. Se quiser, implemente em Go o núcleo do consumo idempotente.

Resposta 3

Temos 3 causas prováveis de eventos duplicados nesse pipeline. ### Primeiro: O produtor sem idempotência pode gerar duplicatas ao reintentar o envio ao broker após um timeout. ### Segundo: O auto-commit com intervalos grandes faz com que o offset seja committado muito depois do processamento — se o consumer crasha, a mensagem já processada é reentregue. ### Terceiro: O consumer lê e processa a mensagem, envia ao antifraude, mas crasha antes de commitar o offset. No rebalance, outro consumer assume a partição, lê a mesma mensagem do último offset commitado, e a reenvia ao antifraude. Para garantir idempotência, atuo em três camadas. No produtor, configuro `enable.idempotence=true` para evitar duplicatas na origem. No consumidor, desativo auto-commit com `enable.auto.commit=false` e uso `commitSync()` apenas após processar e persistir o resultado. Por fim, implemento uma tabela de deduplicação no consumidor indexada por `transaction_id`. Antes de processar cada mensagem, consulto se

aquele `transaction_id` já foi processado. Se sim, descarto a mensagem e comito o offset. Se não, processo, persisto o resultado, registro o id, e então comito. Isso garante que mesmo com reproprocessamento por rebalance, o antifraude nunca recebe o mesmo evento duas vezes. Quanto à ordenação, Kafka garante ordem apenas dentro de cada partição. Usar o `transaction_id` como chave de particionamento garante que todas as mensagens da mesma transação caiam na mesma partição e sejam processadas em ordem. Transações diferentes podem ser processadas em paralelo sem impacto, já que a ordem entre transações distintas não é um requisito de negócio neste cenário.

4 - “Depois do incidente, o que você mudaria para que ele não se repita — ou para detectá-lo em minutos, não em horas? Aponte 2 ou 3 SLIs/alertas que faltavam e o que instrumentaria (logs, métricas, tracing). Considere também backpressure e o comportamento sob 3x de carga.”

Resposta 4

1. SLIs/alertas que faltavam: taxa de crescimento do lag do consumidor (consumer lag rate), latência p99 do downstream, e taxa de erros/duplicatas nos logs do consumidor.
2. Instrumentação: Métricas do Prometheus expostas em todos os serviços, logs estruturados (slog) com `transaction_id` para correlação, e tracing distribuído (OpenTelemetry) para rastrear onde o tempo é perdido.
3. Backpressure: Redis como circuit breaker — se o consumidor detectar N timeouts consecutivos no downstream, publica um sinal no Redis que o produtor lê para pausar envios por X segundos. Alternativa: Kafka consumer com `max.poll.records` dinâmico que reduz quando o lag sobe.

5 - Escreva um resumo curto (como você comunicaria ao time e ao gestor durante e após o incidente) e um mini-postmortem: o que aconteceu, impacto, causa provável e ações. Queremos ver clareza, honestidade técnica e responsabilidade sobre o resultado.

Resposta 5

Resumo durante o incidente

“Estamos com um incidente de sobrecarga no serviço de transações devido a campanha de um parceiro (tráfego 3x acima do normal). Lag de consumo crescente e duplicatas detectadas pelo antifraude. Já escalamos o consumer manualmente e confirmamos que o downstream está saudável. Causa raiz ainda sob investigação. Próxima atualização em 15 minutos.”

Mini-postmortem

- O que aconteceu: Campanha de parceiro elevou tráfego em 3x, saturando o consumer e causando lag crescente e duplicatas no antifraude
- Impacto: Latência p99 de liquidação saltou de 200ms para vários segundos; antifraude recebeu eventos duplicados
- Causa provável: HPA mal configurado (maxReplicas baixo), produtor sem idempotência, e consumer sem deduplicação por `transaction_id`
- Ações tomadas: Escalonamento manual do consumer, bloqueio de rebalanceios com `podAntiAffinity`, idempotência no producer, dedup no consumer via Redis
- Ações futuras: Testes de carga obrigatórios antes de campanhas, revisão do HPA, tracing distribuído, e alertas de lag rate

6 - Explique como você usaria ferramentas de IA para acelerar e melhorar sua execução NESTE cenário: em que tarefas se apoiaria, o que pediria à IA, onde confiaria e onde validaria a saída antes de subir, e quais riscos observaria (alucinação, código inseguro, vazamento de dados sensíveis, LGPD).

Resposta 6

Neste cenário, usaria IA principalmente para leitura e resumo de logs do incidente, validação da saída do `kubectl describe deployment` para identificar rapidamente HPA limitado e eventos de erro, e para uma melhor detecção do incidente como um todo. Confiaria na saída da IA

para o resumo de logs e na leitura do describe, pois a IA identifica padrões mais rápido que a leitura manual. Validaria manualmente antes de subir a escrita de boilerplate de novos manifests para correção do erro, bem como a busca de informações online sobre a campanha do parceiro caso informações adicionais sejam requeridas. Quanto aos riscos, sempre cuidaria para não vazarem credenciais nos prompts, pediria os comandos para prévia execução pelo time antes de aplicar, não exporia secrets ou variáveis de ambiente sensíveis, e daria preferência a modelos de IA previamente contratados pela empresa em vez de modelos online públicos, já que estes são mais seguros e mantêm os dados apenas na rede empresarial, mitigando riscos de LGPD.